

Desarrollo de Sistemas Móviles

Semana 7: ORM/DAO para Backend
Exposed, JPA, Flyway, Jooq y SQLAlchemy

Al finalizar la clase, serás capaz de:

- **Comparar frameworks ORM/DAO 2026:** Exposed, JPA/Hibernate, Jooq, SQLAlchemy; cuándo usar cada uno.
- **Active Record vs Data Mapper:** Entender los dos patrones fundamentales y dónde se ubica cada ORM.
- **Exposed DSL:** Implementar queries type-safe con tablas como objects Kotlin y transaction{ }.
- **JPA + Hibernate 6.4+:** Entidades con @Entity, @Repository y queries por nombre de método.
- **Flyway 10:** Migraciones versionadas con archivos SQL, nunca modificar lo ya ejecutado.
- **Problema N+1:** Identificarlo y resolverlo con JOIN FETCH / eager loading.
- **Repository Pattern + Ports & Adapters:** Aislar el ORM del dominio. Tests sin BD real.
- **platform-api (SQLAlchemy 2.0 async):** Ejemplo real con los mismos patrones en Python/FastAPI.

Un ORM (Object-Relational Mapping) mapea tablas a clases del lenguaje, eliminando SQL manual.

- **Sin ORM:** Escribes SQL a mano, manejas Cursor/ResultSet, mapeas columna por columna.
- **Con ORM (Exposed DSL):** `Productos.selectAll().where { Productos.disponible eq true }.map { it.toDto() }`
- **Cómo funciona el mapeo:** El framework necesita saber qué columna corresponde a qué propiedad, declarado via anotaciones (JPA), objects Table (Exposed) o código generado (Jooq).
- **Ventaja principal:** Código más expresivo, refactoring seguro, menos boilerplate. Desventaja: abstracción que puede ocultar queries ineficientes.
- **Regla práctica:** ORM para CRUD cotidiano; SQL directo / Jooq para queries analíticas complejas o de alto rendimiento.

Los dos patrones fundamentales detrás de todo ORM:

- **Active Record:** La entidad ES su propia persistencia. El objeto sabe guardarse: `producto.save()`, `producto.delete()`. Ej: Exposed DAO, Django ORM, Rails ActiveRecord.
- **Data Mapper:** La entidad es un objeto de dominio puro, sin métodos de BD. Un repositorio separado mueve los datos. Ej: JPA/Hibernate, Exposed DSL, Jooq, SQLAlchemy.
- **¿Cuándo Active Record?** CRUD simple, prototipado rápido, apps pequeñas donde la entidad y la BD están muy acopladas.
- **¿Cuándo Data Mapper?** Proyectos medianos/grandes. Permite testear lógica de negocio sin BD real (mock del repositorio).
- **Regla de oro:** Si necesitas testear sin depender de la BD → Data Mapper + Repository Pattern.

Comparativa de Frameworks ORM/DAO 2026

¿Cuándo usar cada framework?

- **Room (Android):** ORM local para apps Android. Flow<List<T>> reactivo. @Entity, @Dao, @Database.
- **Exposed DSL / DAO (JetBrains):** Backend Kotlin. DSL: tablas como objects, transaction{}. DAO: Active Record para CRUD rápido.
- **Spring Data JPA + Hibernate 6.4+:** Estándar enterprise Java/Kotlin. Queries generadas desde nombres de métodos. @Entity, @Repository.
- **Jooq 3.19+:** Genera código desde el esquema de BD. SQL type-safe: si el esquema cambia, el código no compila.
- **R2DBC:** Acceso reactivo (non-blocking) a BD relacionales. Para backends de alta concurrencia con Kotlin Flow.
- **SQLAlchemy 2.0 (async):** Python/FastAPI. ORM async con session.commit(). Mismo patrón Data Mapper que JPA.

Queries en transaction{}

```
object Productos : Table("productos") {
    val id        = integer("id").autoIncrement()
    val nombre    = varchar("nombre", 200)
    val precio    = decimal("precio", 10, 2)
    val disponible = bool("disponible").default(true)
    override val primaryKey = PrimaryKey(id)
}

object ProductosRepository {
    fun getAll(): List<ProductoDto> = transaction {
        Productos.selectAll()
            .orderBy(Productos.nombre to SortOrder.ASC)
            .map { it.toDto() }
    }

    fun crear(req: CreateProductoRequest): ProductoDto = transaction {
        val id = Productos.insertAndGetId {
            it[nombre] = req.nombre
            it[precio] = req.precio.toBigDecimal()
        }
        getById(id.value)!!
    }

    private fun ResultRow.toDto() = ProductoDto(
        id = this[Productos.id], nombre = this[Productos.nombre], ...
    )
}
```

DatabaseConfig + dependencias en build.gradle.kts

Regla de oro: NUNCA fallbackToDestructiveMigration() en producción.

- **Migration(oldVer, newVer):** Implementa override fun migrate(db: SupportSQLiteDatabase) con los ALTER TABLE o CREATE TABLE necesarios.
- **Registro obligatorio:** .addMigrations(MIGRATION_1_2) en el builder. Room ejecuta la cadena de migraciones automáticamente.
- **Campos nullable en produccion:** latitude: Double? en GpsGoogleEntity evita confundir “sin señal GPS” con coordenada (0.0, 0.0) en el Golfo de Guinea.
- **Ejercicio DemoData:** DemoDataDatabase está en version=3 con migración destructiva. Tarea: escribir MIGRATION_2_3 que agrega isSynced INTEGER NOT NULL DEFAULT 0 a las 4 tablas.
- **exportSchema = true:** Genera archivos JSON con el historial del esquema. Imprescindible para auditorías y testing de migraciones.

```
// build.gradle.kts
implementation("org.jetbrains.exposed:exposed-core:0.54.0" )
implementation("org.jetbrains.exposed:exposed-jdbc:0.54.0" )
implementation("org.postgresql:postgresql:42.7.3" )

object DatabaseConfig {
    fun init(
        url: String = System.getenv("DATABASE_URL")
            ?: "jdbc:postgresql://localhost:5432/mi_db" ,
        user: String = System.getenv("DB_USER") ?: "postgres",
        password: String = System.getenv("DB_PASSWORD") ?: ""
    ) {
        Database.connect(url = url,
            driver = "org.postgresql.Driver" ,
            user = user, password = password)
        transaction {
            SchemaUtils.create(Productos) // solo en desarrollo
        }
    }
}
```



```
@Entity
@Table(name = "productos")
data class ProductoEntity(
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    val id: Long = 0,
    @Column(nullable = false, length = 200) val nombre: String,
    @Column(nullable = false, precision = 10, scale = 2)
    val precio: BigDecimal,
    val disponible: Boolean = true,
    @CreationTimestamp val creadoEn: LocalDateTime = LocalDateTime.now()
)

interface ProductoRepository : JpaRepository<ProductoEntity, Long> {
    fun findByDisponibleTrue(): List<ProductoEntity>
    fun findByCategoriaOrderByNombreAsc(cat: String): List<ProductoEntity>

    @Query("SELECT p FROM ProductoEntity p WHERE p.precio BETWEEN :min AND :max" )
    fun findByRangoPrecio(@Param("min") min: BigDecimal,
        @Param("max") max: BigDecimal): List<ProductoEntity>
}
```

Cada cambio de esquema como archivo SQL versionado

Problemas de SharedPreferences que DataStore soluciona:

- **Bloqueo del hilo principal:** Las operaciones de lectura/escritura pueden generar ANR en dispositivos lentos con archivos grandes.
- **Sin null-safety:** `getBoolean()`, `getString()` pueden retornar null inesperadamente sin advertencia del compilador.
- **Sin soporte para coroutines:** No hay `Flow<T>` integrado. Hay que usar `OnSharedPreferenceChangeListener` (callback hell).
- **Sin verificación de tipos:** Se puede escribir un `Boolean` y leerlo como `String` sin error en compilación.
- **DataStore Preferences:** API Flow-based, type-safe, suspend. Lee con `dataStore.data.map{}`, escribe con `dataStore.edit{}`.
- **Proto DataStore:** Para esquemas complejos con Protobuf. Tipado fuerte y migración de esquema explícitamente definida.

```
-- Estructura de archivos:
-- src/main/resources/db/migration/
--   V1_create_products_table.sql
--   V2_add_stock_column.sql

-- V1_create_products_table.sql
CREATE TABLE productos (
  id          SERIAL PRIMARY KEY,
  nombre      VARCHAR(200)    NOT NULL,
  precio      DECIMAL(10,2)   NOT NULL,
  disponible  BOOLEAN         NOT NULL DEFAULT true,
  creado_en   TIMESTAMP       NOT NULL DEFAULT NOW()
);

-- V2_add_stock_column.sql
ALTER TABLE productos ADD COLUMN stock INTEGER NOT NULL DEFAULT 0;
CREATE INDEX idx_productos_categoria ON productos(categoria);

-- application.properties (Spring Boot)
-- spring.flyway.enabled=true
-- spring.flyway.locations=classpath:db/migration
-- NUNCA modificar un .sql ya ejecutado.
```

Error de compilación si el esquema cambia.

```
// Jooq genera clases que representan las tablas exactamente
val productos = dslContext
    .select(PRODUCTOS.ID, PRODUCTOS.NOMBRE, PRODUCTOS.PRECIO)
    .from(PRODUCTOS)
    .where(PRODUCTOS.DISPONIBLE.isTrue)
    .and(PRODUCTOS.PRECIO.between(
        BigDecimal("100"), BigDecimal("500")))
    .orderBy(PRODUCTOS.NOMBRE.asc())
    .fetch()
    .map { record ->
        ProductoDto(
            id = record[PRODUCTOS.ID],
            nombre = record[PRODUCTOS.NOMBRE],
            precio = record[PRODUCTOS.PRECIO].toDouble()
        )
    }

// Si renombra 'nombre' -> 'titulo' en la BD:
// PRODUCTOS.NOMBRE no compila -> error antes de produccion.
```

El Problema N+1

Error de rendimiento más común al usar un ORM con relaciones.

- **¿Qué es?** 1 query para obtener N productos + N queries adicionales para cargar la categoría de cada uno = N+1 queries totales.
- **Cuándo ocurre:** Al iterar una lista y acceder a una relación con lazy loading. Ej: `productos.forEach { it.categoria.nombre }`.
- **Síntoma:** La app funciona bien con 10 registros, se vuelve lenta con 100, se cae con 1000.
- **Causa raíz:** JPA/Hibernate usa lazy loading por defecto. Cada acceso a una relación dispara una nueva query hacia la BD.
- **Solución, JOIN FETCH:** Una sola query con JOIN trae los datos de ambas tablas: `@Query("SELECT p FROM ProductoEntity p JOIN FETCH p.categoria")`.
- **En Exposed DSL:** No hay lazy loading oculto, el desarrollador siempre escribe el join explícitamente: `(Productos innerJoin Categorías).selectAll()`.

break

- break 10: 25 am
- próxima semana revisión de Avances de proyecto
-



PostgreSQL real en Docker durante los tests - no mocks

¿Cuándo usar WorkManager?

Para tareas diferibles que DEBEN completarse eventualmente.

- **Sincronizar datos en background:** Enviar logs de analytics, subir fotos, limpiar datos temporales.
- **Sobrevive la muerte de la app:** Si el proceso muere, WorkManager retoma el trabajo al reiniciar.
- **Constraints declarativos:** `.setRequiredNetworkType(CONNECTED)`, `.setRequiresBatteryNotLow(true)`, `.setRequiresCharging(false)`.
- **Worker vs CoroutineWorker:** `Worker.doWork()` para trabajo síncrono/bloqueante. `CoroutineWorker` con `suspend doWork()` para llamadas a Room/Retrofit.
- **ExistingPeriodicWorkPolicy.KEEP:** No crea una segunda instancia si el trabajo ya está encolado. Evita duplicados.
- **NO usar para:** Tareas inmediatas (usar `viewModelScope` + `coroutines`) o alarmas precisas (usar `AlarmManager` + `BroadcastReceiver`).

```
@SpringBootTest
@Testcontainers
class ProductoRepositoryTest {
    companion object {
        @Container
        val postgres = PostgreSQLContainer< Nothing>("postgres:16-alpine").apply {
            withDatabaseName( "test_db")
            withUsername( "test")
            withPassword( "test")
        }

        @DynamicPropertySource @JvmStatic
        fun configureProperties(registry: DynamicPropertyRegistry) {
            registry.add( "spring.datasource.url" , postgres::getJdbcUrl)
            registry.add( "spring.datasource.username" , postgres::getUsername)
        }
    }

    @Autowired lateinit var productoRepository: ProductoRepository

    @Test
    fun `guardar y recuperar producto`() {
        val p = ProductoEntity(nombre= "Test", precio=BigDecimal("100"), categoria="TEST")
        val saved = productoRepository.save(p)
        val found = productoRepository.findById(saved.id)
        assertTrue(found.isPresent)
    }
}
```


WorkManager 2.9+ - CoroutineWorker

Aislar el ORM del dominio: el código de acceso a datos no debe filtrarse a la lógica de negocio.

- **Repository Pattern:** Toda la persistencia de una entidad se encapsula detrás de una interfaz con métodos de negocio (findAll, crear, eliminar). La implementación puede ser Exposed, JPA o SQL crudo.
- **Ports & Adapters (Hexagonal):** El dominio define interfaces (puertos). La infraestructura provee implementaciones (adaptadores). Ej: IProductoRepository como puerto, ExposedProductoRepository como adaptador.
- **Regla de dependencia:** api $\rightarrow$ domain $\leftarrow$ infra. El dominio NUNCA importa de infraestructura.
- **Ventaja clave:** En los tests se reemplaza el repositorio real por uno fake sin tocar la lógica de negocio. La suite de tests no necesita una BD.
- **Cómo se inyecta:** Constructor injection en Kotlin/Java. @Autowired en Spring. Depends() en FastAPI. El contenedor conecta el puerto con el adaptador en runtime.
- **platform-api lo usa exactamente así:** IGeoEventRepository (Protocol) en domain/ports.py. GeoEventORMRepository en infra/repositories/ lo implementa.

Backend FastAPI + SQLAlchemy 2.0 async. Los mismos patrones vistos hoy, en Python.

- **api/geo_events_orm.py:** FastAPI Router. Recibe POST /geo-events-orm/ y delega al repositorio.
- **domain/ports.py:** IGeoEventRepository(Protocol), el puerto. Define create, find_by_id, find_all, delete.
- **domain/models/geo_event.py:** GeoEvent (Pydantic BaseModel), el DTO de dominio. model_config = {"from_attributes": True} permite GeoEvent.model_validate(orm_obj).
- **infra/orm/geo_event.py:** GeoEventORM(Base), la entidad ORM. Equivalente a @Entity (JPA) o object Table (Exposed).
- **infra/repositories/geo_event_orm_repo.py:** GeoEventORMRepository, el adaptador. Implementa el puerto usando AsyncSession.
- **Dos implementaciones del mismo puerto:** GeoEventRepository (SQL crudo con text()) y GeoEventORMRepository (ORM). Ambas detrás de IGeoEventRepository.

GeoEventORM (SQLAlchemy) vs GeoEvent (Pydantic DTO)

```
# infra/orm/geo_event.py, Entidad ORM (equiv. a @Entity / object Table)
class GeoEventORM(Base):
    __tablename__ = "geo_events"

    id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    user_id: Mapped[Optional[UUID]] = mapped_column(PG_UUID(as_uuid=True), nullable=True)
    latitude: Mapped[float] = mapped_column(Double)
    longitude: Mapped[float] = mapped_column(Double)
    event_type: Mapped[str] = mapped_column(Text, default="gps_ping")
    recorded_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
                                                    server_default=func.now())

# domain/models/geo_event.py, DTO de dominio (equiv. a ProductoDto)
class GeoEvent(BaseModel):
    id: int
    user_id: Optional[UUID] = None
    latitude: float
    longitude: float
    event_type: str
    recorded_at: datetime
    model_config = {"from_attributes": True} # GeoEvent.model_validate(orm_obj)
```

GeoEventORMRepository: Data Mapper con AsyncSession

Recomendación 2026: Firestore como solución primaria para proyectos nuevos.

- **Estructura de datos:** Firestore: documentos + colecciones flexibles. Realtime DB: árbol JSON rígido, difícil de reestructurar después.
- **Consultas:** Firestore soporta filtros compuestos, ordenamiento y paginación. Realtime DB: solo por clave/valor básico.
- **Offline support:** Firestore: caché automática completa. Realtime DB: soporte básico con `persistDiskCache()`.
- **Escalabilidad:** Firestore: serverless, escala automáticamente. Realtime DB: tiene límites de conexiones simultáneas.
- **Precio:** Firestore cobra por lectura/escritura/almacenamiento. Realtime DB por GB transferido. Firestore es predecible.
- **Realtime DB legacy:** Solo para proyectos existentes. Nuevos proyectos siempre Firestore.

```
class GeoEventORMRepository:
    def __init__(self, session: AsyncSession):
        self._session = session

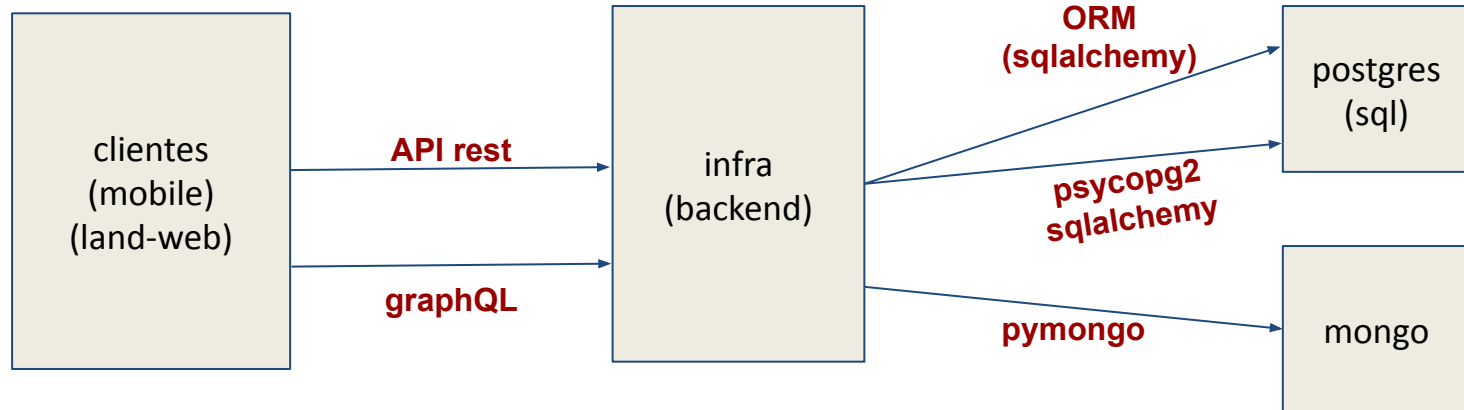
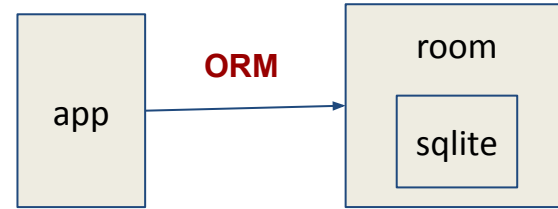
    async def create(self, data: dict) -> GeoEvent:
        event = GeoEventORM(**data)
        self._session.add(event)
        await self._session.commit()
        await self._session.refresh(event)
        return GeoEvent.model_validate(event)

    async def find_all(self, user_id=None, event_type=None,
                      limit=50, offset=0) -> list[GeoEvent]:
        stmt = (select(GeoEventORM)
                .order_by(GeoEventORM.recorded_at.desc())
                .limit(limit).offset(offset))

        if user_id:
            stmt = stmt.where(GeoEventORM.user_id == user_id)
        result = await self._session.execute(stmt)
        return [GeoEvent.model_validate(row) for row in result.scalars()]
```

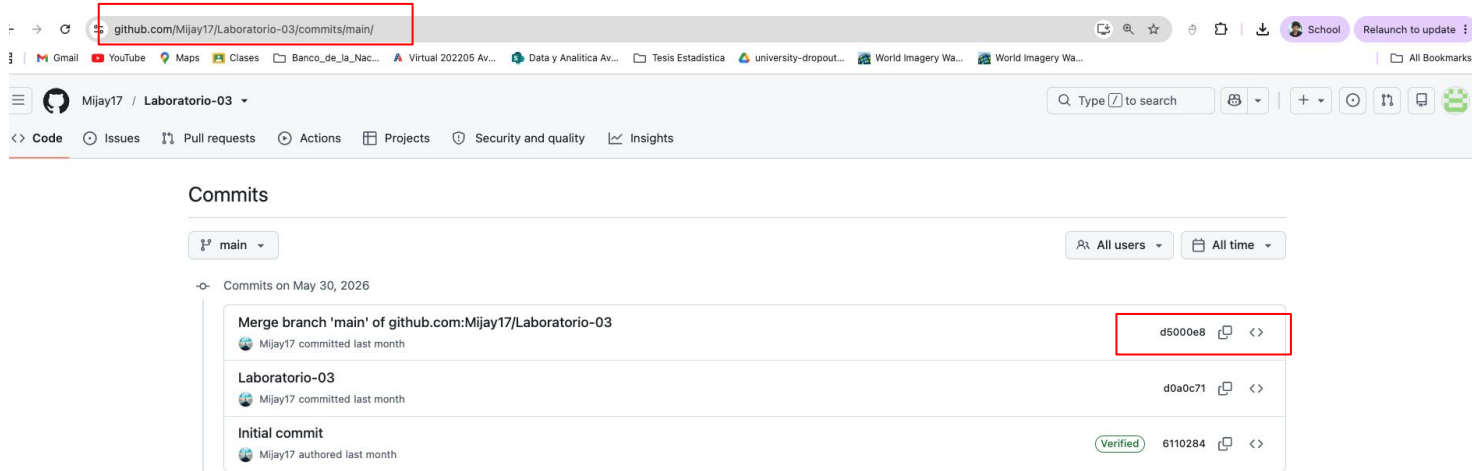
Conceptos clave de esta sesión:

- **ORM vs SQL directo:** ORM: más productivo, menos SQL manual. Jooq: SQL type-safe sin perder control.
- **Active Record vs Data Mapper:** Active Record: la entidad se guarda a sí misma. Data Mapper: repositorio separado. Preferir Data Mapper en proyectos medianos/grandes.
- **Exposed DSL:** Tablas como objects, queries en transaction{}. Type-safe total en Kotlin.
- **JPA + Hibernate:** Estándar enterprise. @Entity, @Repository. Spring Data genera queries desde nombres de métodos.
- **Flyway:** Migraciones como archivos SQL versionados V1_nombre.sql. Nunca modificar migración ya ejecutada.
- **Problema N+1:** 1 query por lista + N queries por relación. Resolver con JOIN FETCH o join explícito en Exposed DSL.
- **Repository + Ports & Adapters:** api → domain ← infra. El dominio define interfaces; la infraestructura las implementa.
- **platform-api:** SQLAlchemy 2.0 async aplica los mismos patrones: GeoEventORM, GeoEvent (DTO), IGeoEventRepository (Protocol).



Por cada laboratorio poner en la últimas columnas **repo_url** (publico) + **commit** (que desea presentar) + **comentario** (opcional) se valora la fecha y cumplimiento del laboratorio

eg.



github.com/Mijay17/Laboratorio-03/commits/main/

Mijay17 / Laboratorio-03

Code Issues Pull requests Actions Projects Security and quality Insights

Commits

main

All users All time

Commits on May 30, 2026

Merge branch 'main' of github.com:Mijay17/Laboratorio-03 Mijay17 committed last month	d5000e8
Laboratorio-03 Mijay17 committed last month	d0a0c71
Initial commit Mijay17 authored last month	Verified 6110284

Lab

Laboratorio, Semana 10 (14)
12:15 pm

Gracias

